

Optimal Resizable Arrays 论文评述

张艺博

徐黄浩

2026 年 7 月 21 日

摘要

本文评述 Robert E. Tarjan 与 Uri Zwick 的论文 *Optimal resizable arrays*。该论文研究如何在保持数组随机访问效率的同时，降低动态扩缩容带来的额外空间开销。本文将围绕问题模型、相关工作、核心数据结构、复杂度证明以及方法局限展开分析。

目录

1	引言与论文定位	4
1.1	研究背景与核心问题	4
1.2	论文定位与评述视角	5
2	问题定义与理论模型	6
2.1	模型定义与复杂度目标	6
2.1.1	可变长数组的抽象接口	6
2.1.2	内存块模型	6
2.1.3	两类空间开销	7
2.1.4	论文的复杂度目标	8
3	文献调研与相关工作	8
3.1	摊还分析与传统方法	8
3.1.1	传统扩缩容策略	9
3.1.2	其他传统工作	9
3.2	理论脉络与下界背景	10
3.2.1	Brodnik 等人的空间下界	10
3.2.2	Tarjan-Zwick 对下界视角的扩展	11
4	论文核心方法	11
4.1	整体构造与技术路线	11
4.1.1	从空间下界到构造目标	11
4.2	空间优化机制	13
4.2.1	$r = 3$ 的两级块结构	13
4.2.2	一般 r 的分层块结构	14
4.2.3	访问时间与均摊更新时间	14
4.2.4	去除稳定空间中的 r 因子	15
5	正确性证明与理论分析	16
5.1	上界分析	16
5.1.1	一般 r 构造的均摊代价	16
5.1.2	访问与修改操作的最坏 $O(1)$ 时间	19
5.2	下界分析	22
5.2.1	标准实现及下界的适用范围	22
5.2.2	(N, k, ℓ) -growth game	23

5.2.3	二项式阈值与游戏下界	23
5.2.4	从真实结构到 $\Omega(r)$ 下界	24
6	技术直觉与方法评价	25
6.1	技术直觉与示例解释	25
6.1.1	技术直觉	25
6.1.2	示例解释	26
6.2	创新点、局限性与后续影响	27
6.2.1	核心创新与理论贡献	27
6.2.2	模型假设与定理边界	28
6.2.3	均摊更新与工程可行性	28
6.2.4	开放问题与后续意义	29
7	总结	29

1 引言与论文定位

1.1 研究背景与核心问题

数组及其可变长实现是程序设计中最常用的数据结构之一。本文讨论的可变长数组允许在末尾添加或删除元素，同时支持按下标访问任意位置 [1]。当已有内存块被填满且其后方空间不可继续使用时，扩容通常需要申请一个更大的内存块，将原有元素复制到新块，再释放旧块。由此产生的核心问题是：能否在保持 $O(1)$ 随机访问和高效扩缩容的同时，使数组占用的空间尽可能接近存放 N 个元素所必需的空间？

Java 的 `ArrayList`、C++ 的 `vector` 等工程实现通常采用连续存储和几何扩容。该方案结构简单且具有良好的局部性，但会长期保留未使用容量，并在重建过程中短暂保存新旧两份表示。因此，即使动态数组已被广泛使用，其空间开销在不同时间尺度上的最优边界仍值得进一步研究。

这一问题可以从**机制**与**策略**两个层面加以分离与理解。

机制层面

1. 系统提供固定大小的存储块 (block)；
2. 存储块可以存放实际数据，也可以存放指向其他块的指针；
3. `Grow` 操作申请新块，并在必要时迁移数据；
4. `Shrink` 操作删除末尾元素，并释放不再需要的空块。

策略层面

1. 块的大小如何选取？
2. 使用多少层指针，`Grow` 过程中块的大小如何变化？
3. `Shrink` 的触发条件是什么，何时进行全局重建 (rebuild)？

将机制与策略解耦，有助于清晰地分析和设计可变长数组的数据结构。

经典方法：翻倍策略 (Doubling) 工程中常用的几何增长策略在数组填满时申请更大的连续内存，并将已有元素复制过去。以翻倍为例，新容量是旧容量的两倍；该策略支持 $O(1)$ 最坏情况随机访问和 $O(1)$ 均摊追加时间 [2]，但稳定空间仍为 $N + O(N)$ 。

更一般地，设增长因子为 $1+\alpha$ 。容量为 C 的数组填满后，新容量变为 $C' = (1+\alpha)C$ 。扩容需要复制 C 个元素，而下一次扩容前至少可以执行约 αC 次追加，因此平均复制代价为 $O(1/\alpha)$ 。另一方面，扩容刚结束时有约 αC 个空位，占新容量的比例为 $\alpha/(1+\alpha)$ ；

翻倍策略对应 $\alpha = 1$ ，此时约一半容量暂未使用。减小 α 可以降低稳定状态下的空闲比例，却会增加扩容频率和均摊复制代价。第 3 章将进一步比较这一时间-空间折中与分块方法。

论文的目标 能否将空间占用压缩至 $N + o(N)$ ，同时仍然保持 $O(1)$ 的随机访问时间？这正是 Tarjan 和 Zwick 论文试图回答的核心问题 [1]。

核心思想 论文的核心建模思想是区分两种空间开销：一是数组长度为 N 时，为存储元素并支持访问而长期保留的稳定空间；二是执行 `Grow` 或 `Shrink` 时，为数据搬移和结构调整而短暂使用的临时空间。

1.2 论文定位与评述视角

上述问题的关键并不只是把总空间从 $O(N)$ 改写成更小的渐近式，而是辨认不同空间开销出现的时间尺度。几何扩容在大多数时刻都保留一段未使用容量；分块结构虽然减少了这部分浪费，却需要长期保存块指针和尾块空位；完整复制则会在扩缩容瞬间同时保留新旧两份表示。若把这些开销全部记入同一个峰值，就无法区分“每个数组都长期承担的成本”和“少数正在扩缩容的数组暂时承担的成本”。因此，论文把稳定额外空间 $s(N)$ 与扩缩容临时空间 $t(N)$ 分开分析，这不仅改变了记账方式，也改变了可被优化的目标。

这种区分在同时维护许多数组的场景中尤其有意义。假设系统中共有 K 个长度约为 N 的可变长数组，而同一时刻只有常数个数组执行扩缩容，那么稳定额外空间会累积为 $K \cdot s(N)$ ，临时空间却不会按相同方式同时累积。允许单次扩缩容短暂使用较大的工作区，可能显著降低所有数组长期驻留的总空间。论文因此没有寻找一个孤立的“最佳数组”，而是刻画一族由参数 r 控制的折中：增大 r 可以压低稳定空间，但会提高临时空间峰值和均摊更新时间。

从评述角度看，判断这一结果是否真正“最优”需要回答三个问题：给出的空间折中能否由具体数据结构达到，访问与扩缩容时间是否仍满足要求，以及是否存在匹配的下界排除更优方案。后文将分别讨论分层块结构和数据结构转换（data structuring transformation）所给出的上界、乘积关系 $s(N)t(N) = \Omega(N)$ 所刻画的空间边界，以及 growth game 导出的标准实现时间下界；同时也会区分论文已经证明的结论与依赖 word-RAM、均摊分析和标准实现（standard implementation）的适用边界。

2 问题定义与理论模型

2.1 模型定义与复杂度目标

本节将第 1 章中机制与策略的区分形式化：申请和释放内存块、通过索引块维持可达性构成底层机制，空间函数的选择和分层构造则属于数据结构策略。下面给出可变长数组（resizable array）的操作接口，以及 Tarjan 和 Zwick 用于衡量空间开销的模型 [1]。与一般动态序列不同，论文研究的对象更接近栈式数组：元素只能在数组末端增加或删除，但任意位置的读取与修改仍需保持常数时间。

2.1.1 可变长数组的抽象接口

设当前数组为 A ，长度为 N 。论文中的可变长数组支持以下基本操作：

1. `Array()`：创建并返回一个初始为空的数组；
2. `Length()`：返回数组当前长度；
3. `Get( $i$ )`：返回下标为 i 的元素，其中 $0 \leq i < N$ ；
4. `Set( $i, a$ )`：将下标为 i 的元素修改为 a ，其中 $0 \leq i < N$ ；
5. `Grow( $a$ )`：在数组末尾加入一个新元素 a ，使数组长度从 N 变为 $N + 1$ ；
6. `Shrink()`：删除数组末尾元素，使数组长度从 N 变为 $N - 1$ 。

这里最重要的限制是：插入和删除只发生在末端。若允许在任意位置插入或删除，则后续元素的下标都可能发生变化，问题会转化为动态序列维护。该类问题即使不要求极端空间效率，也通常需要更复杂的数据结构和更高的操作复杂度。因此，论文将研究对象限定在末端扩缩容的数组模型上。

2.1.2 内存块模型

论文假设底层内存管理系统允许申请和释放任意固定长度的数组块。一个可变长数组的具体实现由若干动态分配的块组成，主要包括三类：

1. **数据块（data block）**：存放数组中的实际元素；
2. **索引块（index block）**：存放指向数据块或其他索引块的指针；
3. **辅助块（auxiliary block）**：存放块数量、块长度、层级计数器等辅助信息。

这种模型抽象了实际系统中的动态内存分配行为。若数组必须始终存放在单个连续内存块中，那么扩容时通常只能重新申请更大的块并复制所有元素；而论文模型允许数组被分散存放在多个块中，再通过索引块维持常数时间访问。这也是后续分层结构能够降低空间浪费的基础。

论文在该抽象中把一个元素或一个指针都视为占用一个机器字，并假设内存管理器可以申请和释放任意长度的固定数组块。为简化时间分析，单次申请或释放被视为 $O(1)$ ；即使长度为 L 的块需要 $O(L)$ 时间完成分配或释放，文中的均摊时间结论仍然成立。除一个作为入口的主索引块外，其余每个块都必须由某个索引块中的指针指向，否则算法无法访问该块。数据结构的空间占用则定义为当前所有已分配块的长度之和，而不是只统计实际存放元素的位置 [1]。

2.1.3 两类空间开销

论文的关键建模创新是区分两类空间：

1. **存储与访问空间**：当数组处于稳定状态、长度为 N 时，为了存储所有元素并支持 `Get` 和 `Set` 所需的空间；
2. **扩缩容临时空间**：执行 `Grow` 或 `Shrink` 时，在元素移动、块合并或块拆分过程中短暂额外占用的空间。

用论文中的记号表示，设 $s(N)$ 和 $t(N)$ 是两个非递减函数。若一个实现满足以下条件，就称其为 $(s(N), t(N))$ 实现 [1]：

$$\text{稳定存储空间} \leq N + s(N),$$

并且在扩缩容操作过程中最多使用：

$$\text{临时峰值空间} \leq N + t(N).$$

这里的 $s(N)$ 衡量平时多出来的空间，例如索引指针、空闲块和辅助信息； $t(N)$ 衡量一次 `Grow` 或 `Shrink` 过程中可能出现的峰值空间。两条不等式统计的都是包含 N 个数据元素在内的总空间上界，因此 $s(N)$ 和 $t(N)$ 分别表示相对于数据本身的稳定冗余与操作期冗余。要求它们非递减，可以在数组长度于一次操作前后相差 1 时统一使用同一渐近上界。传统讨论往往把这两种空间混在一起计算，而 Tarjan 和 Zwick 的核心观察是：在很多应用中，平时占用空间比扩缩容瞬间的临时空间更值得优化。例如，一个程序可能同时维护许多可变长数组，但任意时刻只有少数数组正在扩容或缩容。图 1 概括了两种空间口径的区别。

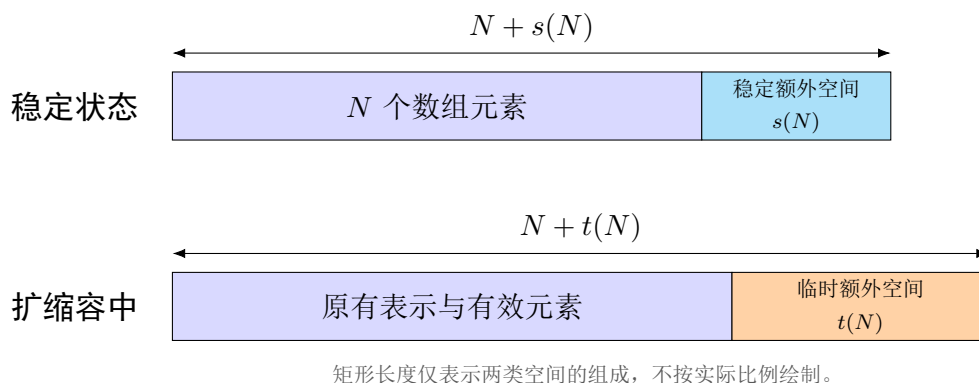


图 1: 稳定空间与扩缩容临时空间的区分（根据文献 [1] 整理）。

2.1.4 论文的复杂度目标

在上述模型下，论文希望同时达到以下目标：

1. `Get` 和 `Set` 操作保持 $O(1)$ 最坏情况时间；
2. `Grow` 和 `Shrink` 操作保持较低的均摊时间；
3. 稳定存储空间尽量接近信息论意义上的下限 N ；
4. 在允许一定临时空间的前提下，突破传统 $N + O(\sqrt{N})$ 空间界限。

论文最终给出的结果是一条由参数 r 控制的折中曲线：对于任意整数 $r \geq 2$ ，可以将稳定存储空间压到 $N + O(N^{1/r})$ ，同时扩缩容时允许短暂使用 $N + O(N^{1-1/r})$ 空间，并保持访问 $O(1)$ 最坏情况时间、扩缩容 $O(r)$ 均摊时间。后文对核心结构和下界证明的分析，都是围绕这组时间-空间折中展开的。

3 文献调研与相关工作

3.1 摊还分析与传统方法

在比较相关工作之前，先简要说明本文反复使用的摊还分析（amortized analysis）。摊还分析考察一段操作序列的总代价，并据此界定每次操作的平均代价；它不依赖输入的概率分布，也不要求每次操作都具有相同的实际成本。常见方法包括：

1. **聚合分析**：直接估计操作序列的总代价，再除以操作次数；
2. **记账法**：为操作预先分配 credit，并用积累的 credit 支付后续高代价操作；
3. **势能法**：用势能函数描述数据结构状态中积累的未来工作量。

本文在分析几何扩容时使用聚合观点，在分析分层块的合并与拆分时使用记账法。两种方法都通过低代价操作积累的余量支付偶尔发生的数据搬移，从而得到 **Grow** 和 **Shrink** 的均摊时间界。

3.1.1 传统扩缩容策略

首先考虑论文中讨论的两类基础策略。

朴素方法 最直接的实现始终维护一个容量恰好等于当前元素数 N 的连续数组，并在每次 **Grow** 或 **Shrink** 时进行全量重建。

该方法的稳定空间为 $N + O(1)$ ，但每次扩缩容都需要复制线性数量的元素，因此均摊更新代价为 $\Theta(N)$ ，操作过程中的峰值空间可达到 $2N + O(1)$ 。

几何扩容 (Geometric Expansion) 另一类传统方法通过预留额外容量减少重建频率。设数组当前容量为 C ，增长因子为 $1 + \alpha$ ；当容量被填满时，算法申请大小为 $(1 + \alpha)C$ 的新数组并复制全部元素。收缩操作通常采用与扩容阈值分离的触发条件，以避免数组长度在边界附近变化时频繁重建。

一次扩容复制 C 个元素，而下一次扩容前至少可完成约 αC 次 **Grow**，因此增长阶段的均摊复制代价为 $O(1/\alpha)$ 。数组容量始终为 $O(N)$ ，但会长期保留线性级别的空闲位置。

增大 α 可以降低扩容频率，却会提高空闲容量比例；减小 α 则产生相反效果。由此可见，单一连续块上的几何扩容只能在线性额外空间与复制频率之间调整常数，无法获得论文追求的 $N + o(N)$ 稳定空间。后续分层分块结构通过更细的空间粒度突破这一限制。

3.1.2 其他传统工作

这里补充论文未详细讨论、但与动态数组设计有关的几类工作。工程中的连续动态数组通常采用几何扩容，不过具体增长因子由标准库实现和版本决定，并不是 C++、Java 或 Python 语言规范统一规定的常数。无论因子如何选择，这类方法的共同点都是保留线性级别的空闲容量，以换取 $O(1)$ 均摊追加时间 [2]。

若改用固定大小为 B 的分块链表，并假设除尾块外的数据块均填满，则块指针和尾部空位带来的额外空间为 $O(N/B + B)$ 。但是，在没有额外目录的情况下，按下标访问需要顺序经过 $O(N/B)$ 个块，不能满足论文要求的 $O(1)$ 最坏情况访问。Bagwell 的 **VList** 将元素放入按几何级数增长的块中，以避免每次增长都整体复制旧元素，并利用块大小的规律支持高效索引 [3]。这类结构说明，分块和几何层次可以同时改善增长与访问，但也会引入多块管理和间接寻址开销。

另一条研究路线考虑比该论文更一般的 rank-based sequence，即允许在任意秩位置插入或删除元素。Goodrich 和 Kloss 提出的 tiered vector 以多层循环数组组织序列 [4]；Bille 等人随后进一步给出了访问、更新和额外空间之间可调的动态数组结构，并进行了实现评估 [5]。这些结果同样使用分层分块思想，但操作模型不同：前者重点处理任意位置更新，Tarjan 和 Zwick 则只允许在末端执行 **Grow** 和 **Shrink**，并进一步区分稳定空间与扩缩容临时空间。

除渐近界之外，Joannou 和 Raman 对多种 extendible array 进行了实证比较，考察其运行时间、空间占用和缓存行为 [6]。这项工作提示我们，较小的渐近额外空间并不必然转化为更快的实际程序；分配次数、局部性和索引层数同样会影响工程表现。

3.2 理论脉络与下界背景

本节梳理可变长数组空间效率问题的前置工作。传统动态数组通过几何扩容获得 $O(1)$ 均摊更新时间，但会保留线性级别的空闲容量 [2]；Sitarski 的 hashed array tree 和 Brodnik 等人的结构则把稳定状态下的额外空间降低到 $O(\sqrt{N})$ [7, 8]。Tarjan 和 Zwick 的工作建立在这条脉络上，但其关键贡献不是简单改进已有结构，而是重新区分“稳定存储与访问空间”和“扩缩容临时空间” [1]。

3.2.1 Brodnik 等人的空间下界

Brodnik 等人的经典结果说明，在不区分临时空间的模型中， $N + \Omega(\sqrt{N})$ 级别的空间占用是不可避免的 [8]。这个下界的证明思路很简洁，也解释了为什么后续工作很难在原有模型下突破 $\sqrt{N}$ 壁垒。

考虑只执行 N 次 **Grow** 操作后的状态。设此时数组元素分布在 k 个连续内存块中。由于这些块共同保存 N 个元素，它们的总大小至少为 N 。同时，为了之后能够访问每个块，数据结构至少需要维护到这些块的指针或等价索引信息。

如果 $k \geq \sqrt{N}$ ，那么仅指针数量就已经造成 $\Omega(\sqrt{N})$ 的额外空间。因此总空间至少为

$$N + \Omega(\sqrt{N}).$$

如果 $k < \sqrt{N}$ ，那么在保存 N 个元素的这些块中，必然存在一个大小超过 $\sqrt{N}$ 的块。设这个大块是在数组长度为 $N' \leq N$ 时被分配的。它刚被分配出来时还没有装入原有元素，而旧的 N' 个元素仍然存放在其他地方，所以当时的总空间会超过

$$N' + \sqrt{N'}.$$

因此，无论块数多还是少，某个时刻都会出现 $N + \Omega(\sqrt{N})$ 级别的空间占用。直观地说，块多会带来指针开销，块少则会带来大块分配时的临时空间峰值。

3.2.2 Tarjan–Zwick 对下界视角的扩展

Tarjan 和 Zwick 并不是否定 Brodnik 等人的下界，而是指出该下界把两类空间混在了一起：一类是平时存储和访问数组需要的空间，另一类是执行 **Grow** 或 **Shrink** 时短暂出现的临时空间 [1]。若用 $(s(N), t(N))$ -implementation 描述二者，则平时总空间为 $N + s(N)$ ，扩缩容过程中的峰值空间为 $N + t(N)$ 。

在这个模型下，Brodnik 下界可以推广为乘积形式。仍然考虑执行 N 次 **Grow** 后的状态。由于每一个数据块都必须可达，块数不能超过额外索引空间的量级，因此可以认为块数至多为 $O(s(N))$ 。这些块一共存放 N 个元素，所以至少有一个块的大小为

$$\Omega\left(\frac{N}{s(N)}\right).$$

当这个大块刚被分配出来时，它尚未存放旧元素，旧元素仍然需要由原有块保存。因此在那次 **Grow** 的过程中，临时额外空间至少达到该大块的大小量级，即

$$t(N) = \Omega\left(\frac{N}{s(N)}\right).$$

于是得到

$$s(N)t(N) = \Omega(N).$$

忽略常数后，这就是论文中 $s(N)t(N) \geq N$ 的含义。它表明平时额外空间和扩缩容临时空间不能同时任意小。若希望稳定存储空间为 $N + O(N^{1/r})$ ，则临时空间至少需要达到 $N + \Omega(N^{1-1/r})$ 。Tarjan 和 Zwick 后续给出的构造正好达到这一量级，因此它在两类空间的折中关系上是最优的。

4 论文核心方法

4.1 整体构造与技术路线

4.1.1 从空间下界到构造目标

在给出具体的构造之前，Tarjan 和 Zwick 首先从已有下界出发，重新审视可变长数组的空间开销。Brodnik 等人的结构达到 $N + O(\sqrt{N})$ 空间，而对应下界表明某些时刻必须使用 $N + \Omega(\sqrt{N})$ 空间。Tarjan 和 Zwick 的关键观察是，这一界限同时计入了稳定状态下长期保留的空间，以及执行 **Grow** 或 **Shrink** 时短暂出现的临时空间；将二者分开后，稳定空间仍有进一步下降的可能。

定理 4.1：平方根下界。 Brodnik 等人的下界可表述为：任何维护可变长数组的数据结构，在某些时刻都必须使用 $N + \Omega(\sqrt{N})$ 空间；即使只考虑 **Grow** 和访问操作，该结论仍然成立 [8, 1]。

其证明对数据块数量分类讨论。假设执行 N 次 **Grow** 后，数组中的 N 个元素分布在 k 个动态分配的连续内存块中。若 $k \geq \sqrt{N}$ ，则为保持每个块可达，数据结构至少需要 k 个指针或等价的索引信息，总空间因而满足

$$N + k \geq N + \sqrt{N}.$$

此时，主要开销来自稳定状态下长期保留的块索引。

若 $k < \sqrt{N}$ ，则根据抽屉原理，至少有一个数据块的大小超过 $\sqrt{N}$ 。设该块在数组长度为 $N' \leq N$ 时被申请。新块刚被分配时尚未承载旧元素，而原有 N' 个元素仍保存在旧块中，因此当时的总空间至少为

$$N' + \Omega(\sqrt{N'}).$$

此时，主要开销来自申请大块时新旧表示短暂共存所产生的临时空间。这两个分支揭示了指针数量与单块大小之间的矛盾，也说明了区分稳定空间和临时空间的必要性。

定理 4.2：乘积下界。 在上述分类思想的基础上，论文将平方根下界推广到任意 $(s(N), t(N))$ -implementation[1]：

$$s(N)t(N) \geq N.$$

其中， $s(N)$ 表示稳定状态下允许的额外空间， $t(N)$ 表示扩缩容过程中允许的临时额外空间。

设稳定总空间至多为 $N + s(N)$ 。扣除保存 N 个元素所需的空間后，指针、索引和块管理信息只能占用 $O(s(N))$ 空间。因此，可达数据块的数量也至多为同一量级。将 N 个元素分布在這些块中，必然存在一个大小为

$$\Omega\left(\frac{N}{s(N)}\right)$$

的数据块。该块被申请时尚未装入旧元素，原有表示仍需保留，因而

$$t(N) = \Omega\left(\frac{N}{s(N)}\right),$$

进而得到 $s(N)t(N) = \Omega(N)$ 。忽略常数后，论文将其写为上述乘积关系；取 $s(N) = t(N) = \sqrt{N}$ 时，即恢复经典的平方根界限。

由下界引出的构造目标。 若希望稳定额外空间满足

$$s(N) = O(N^{1/r}),$$

乘积下界要求临时额外空间至少达到

$$t(N) = \Omega(N^{1-1/r}).$$

因此，后续构造的目标是匹配这一指数折中，即同时实现

$$s(N) = O(N^{1/r}), \quad t(N) = O(N^{1-1/r}).$$

论文先以 $r = 3$ 的两级结构说明基本机制，再推广到一般 r 的多层分块结构； $r = 2$ 时则回到 Brodnik 等人实现的平方根级额外空间。由此，上界构造与乘积下界共同刻画了稳定空间和临时空间之间的渐近最优折中。

4.2 空间优化机制

本节进一步解释上述分层构造中的空间优化机制。核心思想不是消除扩缩容时的数据搬移，而是把空间开销区分为稳定状态下的额外空间和扩缩容过程中的临时空间，并有意识地把较大的空间需求限制在短暂的块合并、拆分和重建过程中 [1]。

4.2.1 $r = 3$ 的两级块结构

论文首先讨论一个 $(O(N^{1/3}), O(N^{2/3}))$ -implementation[1]。令参数 B 满足

$$N^{1/3} \leq B \leq 4N^{1/3}.$$

数据结构使用大小为 B^2 的大块存储数组中已经稳定下来的部分，并使用至多 $2B$ 个大小为 B 的小块保存数组末端。所有大块始终保持填满；小块中至多有一个部分填充的块，并可以额外保留一个空块。两类数据块分别由两个索引块管理，其布局与合并关系如图 2 所示。

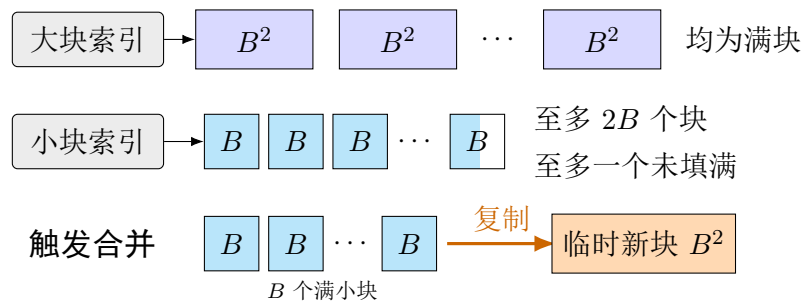


图 2: 参数 $r = 3$ 时的大块、小块与合并过程（根据文献 [1] 重绘）。

这一设计可以看作对 hashed array tree 的再次分层。如果直接使用大小约为 $N^{2/3}$ 的数据块，最后一个数据块可能只存有少量元素，稳定状态下会浪费 $O(N^{2/3})$ 空间。现在改用大小约为 $N^{1/3}$ 的小块表示末端尚未填满的部分，尾部浪费和索引开销都只需 $O(N^{1/3})$ 空间。

这种压缩并非没有代价。当 $2B$ 个小块均已填满时，结构会将其中 B 个小块的元素复制到一个新申请的、大小为 B^2 的大块中；当小块耗尽而数组继续执行 **Shrink** 时，则需要把一个大块拆成 B 个小块。因此，合并或拆分过程中会短暂使用

$$O(B^2) = O(N^{2/3})$$

的额外空间。由此得到的正是“较小稳定空间、较大临时空间”的折中：平时只保留 $O(N^{1/3})$ 额外空间，而扩缩容过程允许 $O(N^{2/3})$ 的临时峰值。

4.2.2 一般 r 的分层块结构

对于任意整数 $r \geq 2$ ，第 6 节把两级结构推广为 $r - 1$ 个层级 [1]。结构维护参数 B ，使得

$$N^{1/r} \leq B < 4N^{1/r},$$

并使用大小为

$$B, B^2, \dots, B^{r-1}$$

的数据块。大小超过 B 的块始终填满；最低层大小为 B 的块中，至多最后一个没有填满。每种块大小对应一个索引块，因此一共维护 $r - 1$ 个索引块。

各层块的数量可以理解为 N 的一种冗余 base- B 表示。若只支持 **Grow**，每层保留至多 B 个块即可：当某层积累了 B 个大小为 B^i 的满块时，将它们合并成一个大小为 B^{i+1} 的块。为了同时支持 **Shrink**，论文把每层的容量放宽到至多 $2B$ 个块。当最低层没有块可供删除时，结构找到最近的非空高层，并将一个高层块逐级拆成更小的块。这个冗余区间使合并与拆分之间留有缓冲，避免连续操作反复触发相反方向的重构。

稳定状态下，数据块除最低层的最后一个块外均为满块，主要额外开销来自各层索引、计数器和少量未使用位置。每层索引规模为 $O(B)$ ，共有 $r - 1$ 层，故稳定额外空间为

$$O(rB) = O(rN^{1/r}).$$

一次合并所申请的最大新块大小为 B^{r-1} ，因此扩缩容过程中的额外空间为

$$O(B^{r-1}) = O(N^{1-1/r}).$$

第 6 节由此得到

$$(O(rN^{1/r}), O(N^{1-1/r}))\text{-implementation.}$$

4.2.3 访问时间与均摊更新时间

分层并不意味着按下标访问必须逐层查找。论文令 B 为 2 的幂，并把各层块数压缩为机器字中的冗余 base- B 计数器。借助 **msb**、**lsb**、位掩码和移位操作，可以在

word-RAM 模型下以 $O(1)$ 最坏情况时间确定目标元素所在的层、数据块及块内偏移。因此，**Get** 和 **Set** 仍保持常数时间 [1]。

合并与拆分虽然可能移动整个块，但其代价可以分摊到一段操作序列上。每个新元素先进入最低层，随后至多沿 $r - 1$ 个层级向上移动；拆分的代价也可以由两次同级拆分之间积累的 **Shrink** 操作支付。因此，**Grow** 和 **Shrink** 的均摊代价为 $O(r)$ 。本章只保留这一结构直觉，完整的 credit 分析和相应下界将在后续理论分析中讨论。

4.2.4 去除稳定空间中的 r 因子

当 r 是常数时， $O(rN^{1/r})$ 与 $O(N^{1/r})$ 在渐近意义下没有区别；但当 $r = r(N)$ 随 N 增长时，第 6 节结果中的 r 因子不能忽略。第 7 节为此提出数据结构转换。它针对一类标准实现：临时空间峰值主要出现在申请新块，并按顺序把若干旧块内容复制到新块的过程中。

设一个标准结构的参数原本为

$$(s(N), O(N)),$$

并给定非递减阈值函数 $t(N)$ 。当它试图申请长度超过 $t(N)$ 的新块时，不再申请单个完整大块，而是顺序申请若干个大小至多为 $t(N)$ 的小块，并保存指向这些小块的额外指针。这样，任一时刻新申请而尚未填满的空间被控制在 $O(t(N))$ ；另一方面，拆分大块所需的额外指针数为 $O(N/t(N))$ 。引理 7.2 给出的转换结果为 [1]

$$\left(s(N) + O\left(\frac{N}{t(N)}\right), s(N) + O(t(N))\right)\text{-implementation,}$$

同时不增加访问操作的渐近最坏情况时间，也不增加扩缩容操作的渐近均摊时间。

为了得到最终界，论文取

$$t(N) = N^{1-1/r},$$

并把该转换应用于第 6 节中参数为 $2r$ 的标准结构。输入结构的稳定额外空间为

$$O(rN^{1/(2r)}).$$

定理 7.3[1] 所处理的参数范围为

$$r \leq \frac{1}{2} \frac{\log N}{\log \log N}$$

在此范围内，上述空间项以及新增的

$$O(N/t(N)) = O(N^{1/r})$$

指针空间都可被 $O(N^{1/r})$ 吸收。最终得到

$$(O(N^{1/r}), O(N^{1-1/r}))\text{-implementation,}$$

并继续支持 $O(1)$ 最坏情况访问和 $O(r)$ 均摊扩缩容时间。这个转换说明，论文的最终空

间界并非只来自增加层数，而是来自“分层表示”与“限制单次物理分配规模”两步优化的结合。

5 正确性证明与理论分析

5.1 上界分析

5.1.1 一般 r 构造的均摊代价

论文引理 6.3 对一般 r 的构造给出了如下均摊分析 [1]:

$$(O(rN^{1/r}), O(N^{1-1/r}))\text{-implementation,}$$

并证明 **Grow** 和 **Shrink** 的均摊代价均为 $O(r)$ 。

上文已经介绍了这一一般构造：令 $B \approx N^{1/r}$ ，数据被存放在大小为

$$B, B^2, \dots, B^{r-1}$$

的多层 blocks 中。每一层最多维护 $2B$ 个 blocks。于是，从空间角度看，每一层的 index block 大小为 $O(B)$ ，一共有 $r - 1$ 层，因此稳定状态下的额外存储空间为

$$O(rB) = O(rN^{1/r}).$$

另一方面，扩缩容过程中最大的临时申请发生在分配一个大小为 B^{r-1} 的数据块时，因此临时空间为

$$O(B^{r-1}) = O(N^{1-1/r}).$$

所以该结构首先满足

$$(O(rN^{1/r}), O(N^{1-1/r}))\text{-implementation}$$

的空间要求。

下面集中说明扩缩容操作的均摊代价。若只考虑连续的 **Grow**，每个新元素从 level 1 向上移动时最多跨越 $r - 1$ 层，因此每个元素承担 $O(r)$ 次复制，这给出了均摊界的初步直觉。但加入 **Shrink** 后，**Split-Blocks** 会把元素移回低层，同一个元素可能再次参与后续合并，因而不能直接用“每个元素只移动 $O(r)$ 次”完成证明。严格分析需要同时为元素和层级分配 credit，并利用两次同级拆分之间必须经过足够多次 **Shrink** 这一事实。

Grow 的均摊代价。 论文将一次 Grow 或 Shrink 的成本定义为元素赋值 (item assignment) 的次数。对于 Grow, 普通插入一个新元素的成本为 1; 若触发 Combine-Blocks, 还需计入合并过程中的元素复制成本。

作者为 level i 中的每个元素分配

$$r - i$$

个 credit。这里 level i 指的是元素当前位于大小为 B^i 的 block 中。新插入的元素总是先进入 level 1, 因此初始获得

$$r - 1$$

个 credit。当某个元素因 Combine-Blocks 从 level i 被复制到 level $i + 1$ 时, 它的 credit 从

$$r - i$$

变为

$$r - (i + 1),$$

正好减少 1, 而这个减少的 credit 就用来支付这一次元素复制的代价。

因此, 每个元素从 level 1 移动到更高层的过程中, credit 始终非负, 且每次复制都由 credit 支付。新插入元素时需要额外分配 $r - 1$ 个 credit, 再加上插入本身的常数成本, 所以 Grow 的均摊代价为

$$O(r).$$

这也对应了论文中“Combine-Blocks 的摊还成本为 0”的结论。

Shrink 的均摊代价。 Shrink 的分析更为细致, 因为它不能简单地反向使用上述元素 credit。该操作始终删除数组末尾元素。若最后一个 size- B block 中仍有元素, 则可直接删除, 元素赋值成本为 0; 若当前不存在 size- B block, 即 $n_1 = 0$, 则必须调用 Split-Blocks, 从更高层拆出低层块后再删除末尾元素。

因此, Shrink 分为两种情况:

1. 最后一个 size- B block 还有元素, 直接删除, 代价为 0;
2. 没有 size- B block, 需要执行一次 Split-Blocks。

设这次 Split-Blocks 发生在 level k , 即当前最小的非空层为 k 。此时算法将最后一个大小为 B^k 的数据块拆开。

拆分后, levels 1 到 $k - 1$ 中共包含 B^k 个元素。size- B^k block 被拆成更低层的数据块: 对 level $k - 1, k - 2, \dots, 2$, 每层各生成 $B - 1$ 个对应大小的块; level 1 最终生成 B

个 size- B blocks。一次拆分因而将一个高层大块展开为多层小块，使数组末尾重新出现可供 **Shrink** 使用的 size- B blocks。

关键观察是：两次同级的 level k 拆分之间，至少要经历 B^k 次 **Shrink**。一次 level k 拆分后，levels $1, \dots, k-1$ 中共有 B^k 个元素；再次触发同级拆分前，必须先删除这些低层元素。若前一次结构变化是 **Combine-Blocks** 创建新的 size- B^k block，则合并后低层同样至少保留 B^k 个元素，因而上述间隔仍然成立。

为支付 **Split-Blocks** 的成本，作者为每次 **Shrink** 额外分配层级 credit：每次操作都给各层增加 3 个 credit。因此，当一次 level k 拆分发生时，该层至少已经积累

$$3B^k$$

个 credit。

以上说明了层级 credit 的来源。一次 **Split-Blocks** 需要支付两部分成本。

第一部分是实际复制成本。拆开一个 size- B^k block，需要复制其中的 B^k 个元素，因此实际 item assignment 成本为

$$B^k.$$

第二部分是给移动后的元素补充 credit。由于 **Split-Blocks** 会把 level k 的元素移动到更低层，而这些元素未来还可能被复制到高层，因此必须补充元素 credit。若一个元素从 level k 移动到 level $i < k$ ，需要额外补充的 credit 数量为

$$k - i.$$

根据 split 后各层元素的分布，需要补充的 credit 总量为

$$B(k-1) + (B-1) \sum_{i=1}^{k-1} (k-i)B^i.$$

其中，第一项与求和式中 $i=1$ 的部分合计为 $B^2(k-1)$ ，对应 level 1 中 B^2 个元素所需补充的 credit；求和式的其余部分对应 level 2 到 level $k-1$ 中的元素。

接下来对该式进行上界估计：

$$B(k-1) + (B-1) \sum_{i=1}^{k-1} (k-i)B^i \leq B \sum_{i=1}^{k-1} (k-i)B^i.$$

令 $j = k - i$ ，则有

$$B \sum_{i=1}^{k-1} (k-i)B^i = B \sum_{j=1}^{k-1} jB^{k-j} \leq B^{k+1} \sum_{j \geq 1} jB^{-j}.$$

利用无穷级数

$$\sum_{j \geq 1} jB^{-j} = \frac{B}{(B-1)^2},$$

可得

$$B^{k+1} \sum_{j \geq 1} j B^{-j} = \frac{B^{k+2}}{(B-1)^2} \leq 2B^k,$$

其中最后一步使用了 $B \geq 4$ 。

因此，一次 level k 的 **Split-Blocks** 所需支付的总成本不超过

$$B^k + 2B^k = 3B^k.$$

在该拆分发生之前，level k 已经至少积累了 $3B^k$ 个 credit，足以支付实际复制成本以及移动元素后的 credit 补充。因此，**Split-Blocks** 的摊还成本为 0。

重建 (Rebuild) 的成本。 最后还需考虑整体重建。**Grow** 或 **Shrink** 可能使参数 B 不再适合当前的 N ，因此算法在 $N = B^r$ 时将 B 翻倍，或在 $N = (B/4)^r$ 时将 B 减半并重建结构。论文同样使用 credit 支付重建成本：每次 **Grow** 或 **Shrink** 为整个数据结构额外增加 $2r$ 个 credit。这样会使每次操作的均摊成本增加 $O(r)$ ，但总界仍保持为 $O(r)$ 。

两次重建之间已经发生线性数量的 **Grow** 或 **Shrink**，因此数据结构能够积累足够的 credit。重建的实际成本为 N ，重建后为所有元素重新分配 credit 的总量至多为 $(r-1)N$ ；积累的全局 credit 足以支付这两部分开销，故重建的摊还成本也可视为 0。

综上，为每次 **Grow** 或 **Shrink** 计入 $O(r)$ 个 credit，足以支付元素插入、块合并、块拆分和周期性重建的成本，因此两种操作的均摊代价均为 $O(r)$ 。

5.1.2 访问与修改操作的最坏 $O(1)$ 时间

论文引理 6.4 说明了如何实现访问与修改操作的最坏情况 $O(1)$ 时间 [1]。一个直接的朴素思路是从高层到低层扫描，判断待访问元素落在哪一层。由于一般 r 的构造中最多有 $r-1$ 层，这样做的复杂度为 $O(r)$ 。这一方法虽然直观，但还不能达到论文要求的最坏情况 $O(1)$ 。因此，作者进一步利用广义 B 进制表示，将“寻找元素所在层”这一过程压缩到常数时间。

具体来说，令 n_0 表示最后一个部分填充的 size- B block 中的元素个数；令 n_1 表示 full size- B blocks 的数量；更一般地， n_j 表示 full size- B^j blocks 的数量。其中 $0 \leq n_0 < B$ ，而对 $1 \leq j < r$ 有 $0 \leq n_j \leq 2B$ 。于是当前数组长度可以写成一种冗余的 base- B 表示：

$$N = \sum_{j=0}^{r-1} n_j B^j.$$

这里之所以称为冗余表示，是因为普通 base- B 表示中每一位应小于 B ，而表示完整块数量的 n_j 可以超过 $B-1$ 。这一放宽不会破坏访问过程，反而正是冗余 base- B 计数器的体现。

接下来定义

$$N_k = \sum_{j=0}^{k-1} n_j B^j.$$

直观上, N_k 表示低于 level k 的各数据块中一共存放了多少元素。由于论文的数据结构以较高层块存放数组前部较稳定的元素, 并以较低层块存放靠近末尾的新元素, 因此判断目标元素所在层可以等价地转化为从数组末尾反向计数。

设要访问的元素下标为 i , 其中 $0 \leq i < N$ 。令

$$x = (N - 1) - i.$$

这里 x 表示该元素从数组末尾倒数的位置。于是, 第 i 个元素位于 level k 当且仅当

$$N_k \leq x < N_{k+1}.$$

这个式子是访问证明中的核心。它说明, 一个元素在哪一层, 并不是由前面有多少元素决定, 而可以由它后面有多少元素决定; 而从后往前看时, 低层 blocks 正好排在前面, 所以判断会更自然。

一旦找到了这个 k , 定位元素就非常容易。从数组正向下标看, level k 对应的区间从 $N - N_{k+1}$ 开始, 因此令

$$y = i - (N - N_{k+1}) = N_{k+1} - 1 - x.$$

则它所在的 block 编号和 block 内偏移分别为

$$\left\lfloor \frac{y}{B^k} \right\rfloor, \quad y \bmod B^k.$$

当 $k = 0$ 时, 元素位于唯一的部分填充 size- B block 中, 偏移就是 y 。由于论文中取 $B = 2^b$, 所以 B^k 也是二的幂, 上述除法和取模都可以用 shift 和 mask 在常数时间内完成。因此, 真正困难的地方只剩下一个: 如何在 $O(1)$ 时间内找到满足 $N_k \leq x < N_{k+1}$ 的层数 k 。

为此, 作者首先将 x 写成标准 base- B 表示, 并令 ℓ 为其最高非零位。当 $x = 0$ 时, 目标是数组最后一个元素, 可以通过后述的非空层位图直接处理; 下面只考虑 $x > 0$ 。由于 $B = 2^b$, 最高非零位可以通过机器字上的最高有效位操作得到:

$$\ell = \left\lfloor \frac{\text{msb}(x)}{b} \right\rfloor.$$

这里 $\text{msb}(x)$ 表示 x 的最高有效 bit 的位置。也就是说, ℓ 大致刻画了 x 的数量级。

接下来需要说明, 找到 ℓ 后, 真正的层数 k 不会离 ℓ 太远。当 $\ell = 0$ 时, $N_0 = 0$, 只需比较 x 与 N_1 ; 若 $x < N_1$, 则 $k = 0$, 否则利用后述位图寻找最小的非空高层。以下设 $\ell \geq 1$ 。论文利用每个 $n_j \leq 2B$ 这一性质, 得到

$$N_{\ell-1} = \sum_{j=0}^{\ell-2} n_j B^j \leq 2B \sum_{j=0}^{\ell-2} B^j \leq 3B^{\ell-1} \leq B^\ell \leq x.$$

因此有

$$x \geq N_{\ell-1},$$

从而可以推出

$$k \geq \ell - 1.$$

这一步非常关键。它说明我们不需要从所有层开始找，而是只需要从 $\ell - 1$ 附近开始判断。

此时可以分三种情况讨论。第一，如果

$$x < N_{\ell},$$

那么结合 $N_{\ell-1} \leq x$ ，可知

$$N_{\ell-1} \leq x < N_{\ell},$$

因此

$$k = \ell - 1.$$

第二，如果

$$N_{\ell} \leq x < N_{\ell+1},$$

那么直接得到

$$k = \ell.$$

第三，如果

$$N_{\ell+1} \leq x,$$

说明 x 已经越过了 level ℓ 对应的范围。此时需要继续向更高层寻找下一个非空层。也就是说， $k = \ell'$ ，其中 $\ell' > \ell$ ，并且 ℓ' 是满足 $n_{\ell'} > 0$ 的最小下标。

最后的问题就是：如何在 $O(1)$ 时间内找到这个下一个非空层。由于表示完整块数量的 n_j 可能达到 $2B$ ，论文将它拆成两部分：

$$n_j = n_j^0 + Bn_j^1,$$

其中

$$0 \leq n_j^0 < B, \quad 0 \leq n_j^1 \leq 2.$$

然后将所有低位部分打包成一个机器字：

$$N^0 = (n_{r-1}^0, \dots, n_1^0, n_0^0)_B,$$

再将所有高位部分打包成另一个机器字：

$$N^1 = (n_{r-1}^1, \dots, n_1^1, n_0^1)_B.$$

在论文的参数范围内, $rb = O(\log N)$, 所以这两个打包结果各自只占常数个机器字。某一层 j 非空当且仅当 $n_j^0 > 0$ 或 $n_j^1 > 0$ 。因此, 只要观察 $N^0 \vee N^1$ 中对应 digit 是否非零, 就可以判断某一层是否非空。

为了找到 ℓ 以上的第一个非空层, 只需要用 mask 去掉低于等于 ℓ 的部分, 然后找剩余部分中的最低有效位即可。由于最低有效位 lsb 也可以通过机器指令或由 msb 间接得到, 这一步也可以在常数时间内完成。由此, 作者成功在 $O(1)$ 时间内找到了目标元素所在的 level k 。

得到 k 后, 再根据 N_{k+1} 计算 $y = N_{k+1} - 1 - x$, 并用移位和掩码得到数据块编号及块内位置, 即可直接访问该元素。修改操作使用相同的定位过程, 只需在定位后替换对应位置的值。因此, **Get** 和 **Set** 均可支持最坏情况 $O(1)$ 时间。

总的来说, 朴素扫描层级需要 $O(r)$ 时间, 而论文通过把各层块数编码为冗余 base- B 表示, 再利用机器字上的 msb、掩码和移位等操作, 将寻找目标层的过程压缩到常数时间。访问与修改能够达到 $O(1)$ 的关键并非多层结构本身, 而是把层定位转化为机器字上的位运算问题。

5.2 下界分析

第 3 章已经说明, 若稳定状态下的额外空间为 $s(N)$, 扩扩容过程中允许的临时额外空间为 $t(N)$, 则二者满足

$$s(N)t(N) = \Omega(N).$$

这一结论刻画的是空间之间的权衡。本节转而讨论更新时间: 当稳定额外空间被压缩到 $O(rN^{1/r})$ 时, **Grow** 的均摊代价是否还可能低于 $O(r)$ 。论文第 8、9 节通过 growth game 证明, 在一类标准实现中答案是否定的 [1]。

5.2.1 标准实现及下界的适用范围

论文把块迁移遵循特定形式的结构称为标准实现。每当结构申请一个新块 B 时, 它立即将若干旧块 $A_1, A_2, \dots, A_q$ 中的元素按顺序复制到 B ; 一个旧块复制完毕后随即释放。此次合并结束后, 结构重新回到只占用 $N + s(N)$ 空间的稳定状态。因而, 临时空间只在“申请新块并搬移旧块”的过程中短暂存在。

这一限制使真实的块合并能够被组合游戏刻画。论文前面给出的分层结构以及经典分块结构都属于此类实现, 但该定义并不直接覆盖把一次迁移长期分散到许多操作、使用更复杂编码或不遵循上述复制方式的算法。因此, 后面的时间下界必须连同标准实现这一前提一起理解。

5.2.2 (N, k, ℓ) -growth game

growth game 依次插入 N 个元素，并用 k 个初始为空的子数组 $A_1, A_2, \dots, A_k$ 保存它们。空子数组位于序列前部；第一个非空子数组至多保留 ℓ 个空位，其余非空子数组均为满块。若 a_i 表示 A_i 中的元素数，则一次插入分为三种情况：

1. 第一个非空子数组仍有空位，直接写入新元素，代价为 1；
2. 所有非空子数组都已填满但仍有空子数组，申请一个大小为 $\ell + 1$ 的新块，写入一个元素并留下 ℓ 个空位，代价为 1；
3. 所有 k 个子数组都非空且已填满，选择某个 $i \in [k]$ ，将 $A_1, \dots, A_i$ 与新元素合并成新的 A_i ，并令前 $i - 1$ 个子数组重新为空。其代价为

$$1 + \sum_{j=1}^i a_j,$$

即一次新元素写入加上所有被搬移旧元素的赋值次数。

图 3 展示了选择 $i = 2$ 时，新元素与前两个子数组合并的过程。

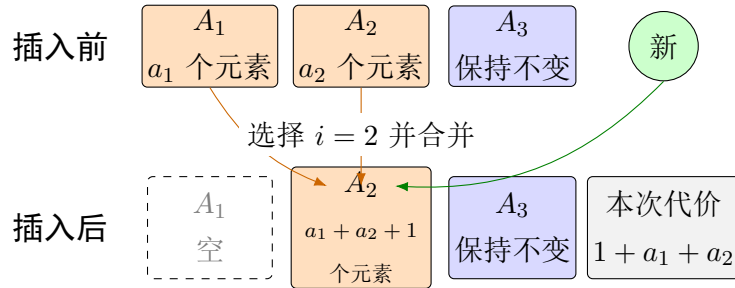


图 3: growth game 中选择 $i = 2$ 的一次合并示例（根据文献 [1] 整理）。

游戏忽略内存申请、释放和指针更新的成本，也不限制一次操作的临时空间，因此它给算法提供了比真实模型更宽松的条件。记完成游戏的最小总代价为 $C_{N,k,\ell}$ ，对应的最小均摊代价为

$$A_{N,k,\ell} = \frac{C_{N,k,\ell}}{N}.$$

5.2.3 二项式阈值与游戏下界

引理 8.2 首先消去空闲位置参数 [1]。当 N 能被 $\ell + 1$ 整除时，连续的 $\ell + 1$ 次插入可视为一次成组插入，从而

$$C_{N,k,\ell} = (\ell + 1)C_{N/(\ell+1),k,0}, \quad A_{N,k,\ell} = A_{N/(\ell+1),k,0}.$$

因此，允许一个块保留少量空位只会缩短等价游戏的长度，并不会从根本上改变均摊搬移代价。以下简记 $C_{N,k} = C_{N,k,0}$ 和 $A_{N,k} = A_{N,k,0}$ 。

定理 8.6 精确求解了 $\ell = 0$ 的游戏 [1]。若整数 $n \geq 0$ 满足

$$\binom{n+k-1}{k} - 1 \leq N \leq \binom{n+k}{k} - 1,$$

则

$$C_{N,k} = (N+1)n - \binom{n+k}{k+1}.$$

这里的 n 可以看作最优策略不可避免的合并深度：当插入数量跨过二项式阈值时，元素必须经历更多层合并。该定理的完整证明通过归纳刻画所有最优终态；对于后续下界，更关键的是推论 8.13：若

$$\binom{n+k-1}{k} \leq N \leq \binom{n+k}{k} - 1,$$

则

$$A_{N,k} \geq \frac{k}{k+1}(n-1) \geq \frac{n-1}{2}.$$

换言之，只要游戏规模越过参数 n 对应的二项式阈值，就能得到 $\Omega(n)$ 的均摊代价。

5.2.4 从真实结构到 $\Omega(r)$ 下界

考虑稳定状态只使用

$$N + O(rN^{1/r})$$

空间的标准可变长数组，并令 $m = rN^{1/r}$ ；常数因子可通过微调参数吸收。稳定额外空间限制了结构能够同时维护的块数、块指针数和空闲位置，因此其操作可以映射到扩展的 (N, m, m) -growth game。扩展规则甚至允许合并任意一组子数组。引理 8.17 和 8.18 说明这些额外自由不会降低最优总代价，所以该游戏覆盖了标准实现可能采用的块合并方式。

利用引理 8.2，并忽略不影响渐近结果的整除和取整问题，可将游戏约化为

$$N' = \Theta\left(\frac{N}{m}\right) = \Theta\left(\frac{1}{r}N^{1-1/r}\right)$$

次成组插入、 $k = m = rN^{1/r}$ 个子数组且 $\ell = 0$ 的游戏。现在取 $n = \beta r$ ，其中 $\beta = 1/2$ 。由标准二项式估计

$$\binom{x}{y} \leq \left(\frac{ex}{y}\right)^y$$

可得

$$\binom{\beta r + rN^{1/r}}{\beta r} \leq \left(\frac{e(\beta + 1)}{\beta}\right)^{\beta r} N^\beta.$$

当 $r = O(\log N)$ 且 N 充分大时，取合适的常数即可使该上界小于 N' 。例如 $\beta = 1/2$ 时，证明中使用

$$r(3e)^{r/2}N^{1/2} \leq N^{3/4} \leq N^{1-1/r}.$$

因此，约化后的游戏已经越过 $n = \Theta(r)$ 对应的二项式阈值。由推论 8.13，

$$A_{N',k} \geq \frac{n-1}{2} = \Omega(r).$$

由此得到定理 9.1：任何稳定存储空间为 $N + O(rN^{1/r})$ 、且 $r = O(\log N)$ 的标准可变长数组，其 **Grow** 操作的均摊代价均为 $\Omega(r)[1]$ 。

第 6、7 节构造达到 $O(r)$ 均摊扩缩容时间，因此该上界在标准实现范围内是渐近最优的。特别地，第 7 节最终结构只使用更小的 $O(N^{1/r})$ 稳定额外空间，自然也满足上述空间限制。需要强调的是，这里证明的是标准实现的均摊下界，而不是所有可能算法的无条件下界，也不能直接解释为一般的最坏情况时间不可能性；将结论推广到非标准算法仍是论文留下的问题。

6 技术直觉与方法评价

6.1 技术直觉与示例解释

6.1.1 技术直觉

在这项工作之前，Brodnik 等人的经典下界已经指出，可变长数组在某些时刻需要 $N + \Omega(\sqrt{N})$ 空间。Tarjan 和 Zwick 并未否定这一结论，而是进一步区分稳定额外空间与扩缩容临时空间，从而揭示更一般的时间-空间关系 [1]。

第一个技术直觉是平方根界限只是折中曲线上的一个平衡点。两类额外空间满足

$$s(N)t(N) = \Omega(N).$$

当二者规模相同时， $s(N) = t(N) = \Theta(\sqrt{N})$ ，正好恢复 Brodnik 等人的平方根级界限；若把稳定额外空间降至 $O(N^{1/r})$ ，则扩缩容临时空间必须上升到 $\Omega(N^{1-1/r})$ 。因此，“空间最优”并非单一数值，而取决于稳定驻留开销与操作期峰值采用何种权衡。

第二个技术直觉是数组中不同位置的元素具有不同的稳定程度。由于 **Grow** 和 **Shrink** 只作用于数组末尾，靠近末尾的元素更新频繁，而靠近开头的元素相对稳定。多层分块结构利用了这一差异：新元素先进入较小的低层块，便于处理尾部变化；随着元素远离末尾，再逐步合并到更大的高层块中，以减少块数量和指针开销。

大小为 $B, B^2, \dots, B^{r-1}$ 的数据块对应不同的稳定层次。Level 1 的 size- B blocks 变化最频繁，类似计数器的低位；更高层的数据块只有在低层积累到阈值后才会变化，类

似计数器的高位。冗余 base- B 计数器因而不仅用于记录块数，也刻画了元素从不稳定到稳定的迁移过程。

第三个技术直觉是冗余区间提供了防止反复重构的缓冲。若一层达到 B 个块就立即合并，并在稍低于阈值时立即拆分，则交替的 Grow 和 Shrink 可能造成合并与拆分反复发生。论文允许每层保留至多 $2B$ 个块；达到 $2B$ 个块时，只把其中 B 个合并到上一层，并在当前层保留另外 B 个。若该合并生成 size- B^k block，则低层仍至少保留 B^k 个元素，在下一次同级拆分前必须先执行至少 B^k 次 Shrink。这些操作积累的 credit 足以支付后续拆分和 credit 补充成本，从而保证 $O(r)$ 均摊更新时间。

6.1.2 示例解释

下面用 $r = 4$ 、 $B = 3$ 的小规模状态说明一次合并。这里选择 $B = 3$ 是为了让图中块数和元素数量保持可读；数据结构的操作规则不变，而引理 6.3 中用于估计 credit 的常数不等式仍按正式证明所要求的 $B \geq 4$ 处理 [1]。

在插入元素 19 之前，level 1 中已有 $2B = 6$ 个填满的 size- B blocks，依次存放元素 1 到 18。此时继续执行 $\text{Grow}(19)$ 会触发 Combine-Blocks：首先把最旧的前 $B = 3$ 个小块，即元素 1 到 9，复制到一个新的 size- B^2 block；随后释放这三个旧块，并把仍存放元素 10 到 18 的三个小块保留在 level 1；最后申请新的 size- B 尾块，将元素 19 放入其中。操作结束后的状态如图 4 所示。

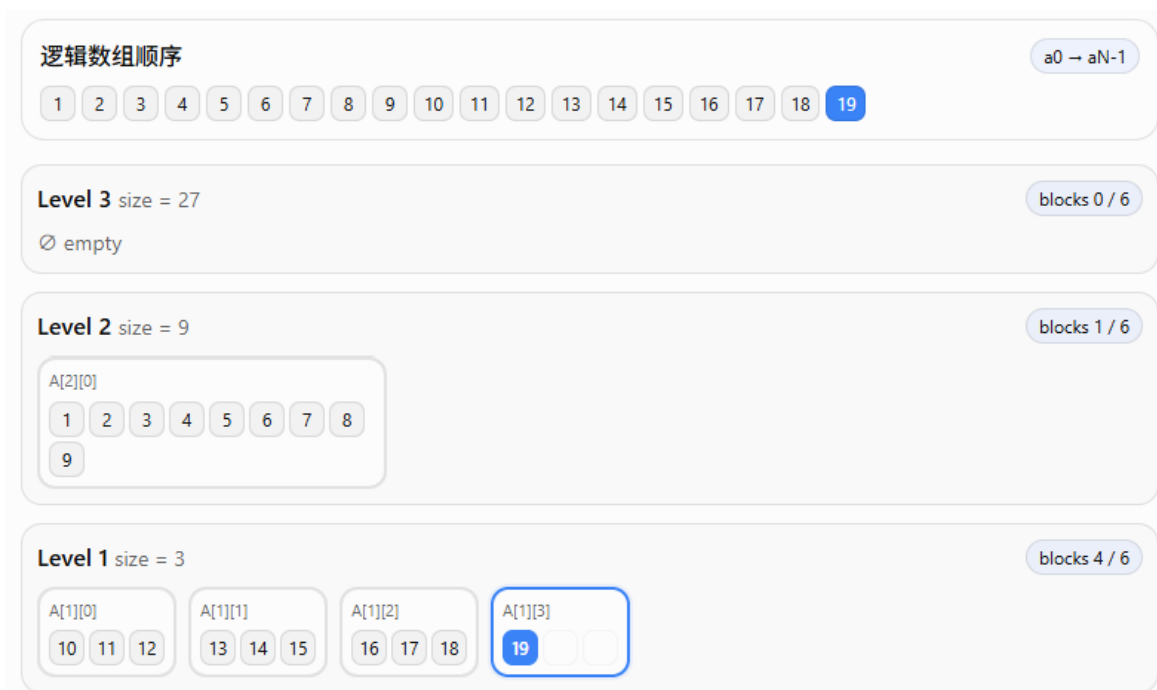


图 4: $r = 4$ 、 $B = 3$ 时执行 $\text{Grow}(19)$ 后的分层状态。元素 1-9 已合并到 level 2，较新的元素仍保留在 level 1。

这个例子同时解释了为何结构总是合并较旧的块。数组只在末尾更新，元素 1 到 9 短期内不会被 **Shrink** 删除，适合放入更大的块以减少指针开销；元素 10 到 19 更接近数组末尾，继续留在小块中可以降低后续尾部变化的重组成本。合并过程中，新 $\text{size-}B^2$ block 与被复制的旧块短暂共存，其 B^2 空间计入扩缩容临时空间；操作结束并释放旧块后，长期保留的主要冗余重新变为各层索引和少量尾部空位。

Shrink 使用相反方向的结构变化，但并不是每删除一个元素就立即撤销合并。当 level 1 中已经没有 $\text{size-}B$ block 可供删除时，算法寻找最低的非空高层，并对该层最后一个块执行 **Split-Blocks**。例如，一个 $\text{size-}B^2$ block 会被拆成 B 个 $\text{size-}B$ blocks，使末尾元素重新暴露在最低层。由于一次合并后 level 1 仍保留 B 个满块，在再次触发同级拆分之前必须先经过至少 B^2 次 **Shrink**；这个缓冲区正是前文 credit 分析能够摊还复制成本的直观原因。

6.2 创新点、局限性与后续影响

前文已经分别介绍了论文的分层构造和下界证明。本节不再重复具体操作，而是从模型、理论完整性和实际可实现性三个角度评价其贡献与边界 [1]。

6.2.1 核心创新与理论贡献

该论文最重要的贡献首先是重新定义了“空间最优”的讨论方式。传统的 $N + \Omega(\sqrt{N})$ 下界把稳定状态下的存储空间和单次扩缩容过程中短暂出现的临时空间合并计算，容易使人误以为两者都不能低于 $\sqrt{N}$ 。Tarjan 和 Zwick 将它们分别记为 $s(N)$ 和 $t(N)$ ，并证明了如下乘积下界 [1]：

$$s(N)t(N) = \Omega(N).$$

这一模型揭示的不是单一最优点，而是一条空间折中曲线：只要允许扩缩容时短暂申请更大的块，稳定空间就可以进一步下降。

其次，论文把这一模型转化为可达到的参数化结果。对整数参数 r ，分层块结构与数据结构转换共同实现

$$s(N) = O(N^{1/r}), \quad t(N) = O(N^{1-1/r}),$$

同时保持 $O(1)$ 最坏情况访问和 $O(r)$ 均摊扩缩容时间。 r 因而不只是证明中的层数，也直接表达设计者在稳定空间、临时峰值和更新代价之间的选择。与只给出某个固定结构相比，这一族构造更完整地描述了问题的可行区域。

最后，论文没有停留在上界构造，而是引入 growth game 对块合并过程进行抽象，并精确求解游戏的最优总代价。由此得到的 $\Omega(r)$ 均摊下界与构造的 $O(r)$ 上界匹配。空间乘积下界和 growth game 时间下界分别封闭了折中关系的两个方向，使“最优”不仅是对某个算法分析得到的结论，而是在明确模型中的渐近最优性结论。

6.2.2 模型假设与定理边界

上述最优性依赖若干需要明确保留的前提。首先， $O(1)$ 访问建立在 word-RAM 模型上。第 6.3 节把各层块数压入少数机器字，并借助 **msb**、**lsb**、位掩码和移位在常数时间定位目标层；常数时间计算 **msb** 的一种方案还依赖乘法。作者也指出，对实际中预期很小的 r ，直接扫描 $O(r)$ 层或进行 $O(\log r)$ 二分可能比位级实现更合适 [1]。因此，这里的常数访问时间主要完善了理论界，并不自动意味着对应实现具有更低的工程开销。

其次，最终的定理 7.3 并非对任意增长速度的 $r(N)$ 都直接成立，其参数范围为

$$r \leq \frac{1}{2} \frac{\log N}{\log \log N}.$$

该定理还需要把第 6 节参数为 $2r$ 的标准结构作为转换输入 [1]。若只陈述最终复杂度而省略范围和参数替换，会把论文实际证明的结果扩大。

更关键的限制来自时间下界。定理 9.1 只覆盖标准实现 (standard implementations)，即新块申请后立即复制旧块并在操作结束时恢复稳定空间界的实现。作者相信，在对元素和指针加入适当的不可压缩性假设后，下界可以推广到所有算法，但论文没有给出这一推广的形式化证明 [1]。因此，“非标准算法不会更好”属于作者的判断，而不是论文已经证明的定理。

6.2.3 均摊更新与工程可行性

论文的 **Grow** 和 **Shrink** 时间是 $O(r)$ 均摊界，而不是最坏情况界。对此也应区分已证明的下界和结构本身的空间障碍：定理 9.1 的 $\Omega(r)$ 均摊下界并不排除一个假想的 $O(r)$ 最坏情况算法；现有构造难以直接去摊还化，是因为常规后台重建 (background rebuilding) 会改变空间的归类。

在最高层合并中，新块规模可达

$$\Theta(N^{1-1/r}).$$

若复制在一次操作内完成，该块只计入临时空间；若把复制分散到多个操作，新旧表示会跨操作同时保留，未完成的新块便必须计入稳定空间。当 $r > 2$ 时，

$$N^{1-1/r} > N^{1/r},$$

所以常规后台重建需要持久保存的块可能超过 $O(N^{1/r})$ 的稳定额外空间预算。这说明标准后台重建不能直接用于第 6、7 节结构，但不能据此断言所有非标准去摊还化方案都不存在。 $r = 2$ 时两种规模同为 $\Theta(\sqrt{N})$ ，也与 HAT 和 Brodnik 等经典结构能够获得最坏情况更新界的事实一致 [7, 8]。

从实现角度看，多层块目录、重建阈值、冗余 base- B 计数器和位操作都会带来额外常数与维护复杂度。论文的优势主要体现在稳定额外空间极为受限的理论场景；在一

般程序中，连续内存的缓存局部性、分配器成本和实现复杂度可能使简单的几何扩容或较浅的分块结构更有吸引力。这不削弱渐近结果，但说明理论最优与工程选择采用的是不同评价标准。

6.2.4 开放问题与后续意义

论文结尾明确留下两个理论问题 [1]。第一，将 $\Omega(r)$ 均摊下界从标准实现推广到所有的可能的可变长数组算法，并为元素与指针的不可压缩性建立合适模型。第二，growth game 的精确解虽然完整，但证明较长；是否存在不求精确值、只用更短的归纳或势能论证就能推出所需渐近下界，仍值得研究。

从评述角度，还可以进一步考察固定小参数下的实际表现，例如比较 $r = 2$ 、 $r = 3$ 分层结构与 HAT、几何扩容数组在缓存行为、分配次数和常数开销上的差异。无论这些结构是否直接进入工程实现，该论文都改变了问题的理论视角：它把“最省空间的动态数组”从寻找单个结构，转化为刻画稳定空间、临时空间和更新时间三者的完整可达边界；growth game 也提供了一种分析受限块合并策略的独立组合工具。

7 总结

Tarjan 和 Zwick 的工作重新审视了可变长数组中“空间最优”的含义。传统分析通常只记录扩缩容期间出现过的最大空间占用，因此得到 $N + \Theta(\sqrt{N})$ 的经典界限；论文则把稳定状态下用于存储和访问的额外空间 $s(N)$ ，与一次 Grow 或 Shrink 中短暂使用的临时空间 $t(N)$ 分开计算。这个模型更准确地反映了同时维护许多数组、但只有少数数组正在扩缩容的场景，也把问题从寻找单一最优结构转化为刻画稳定空间、临时空间和更新时间之间的完整折中关系 [1]。

在上界方面，论文以整数参数 r 控制分层深度，通过大小为 $B, B^2, \dots, B^{r-1}$ 的数据块、冗余 base- B 计数器和后续的数据结构转换，得到稳定存储空间 $N + O(N^{1/r})$ 、扩缩容峰值空间 $N + O(N^{1-1/r})$ 的可变长数组。该结构同时支持 $O(1)$ 最坏情况时间的 Get 和 Set，以及 $O(r)$ 均摊时间的 Grow 和 Shrink。 $r = 2$ 对应经典的平方根级额外空间；增大 r 则能够进一步压缩长期保留的空间，但需要更深的分层和更高的均摊更新代价。

论文还从空间和时间两个方向说明这些结果的最优性。对一般实现，块数量与大块分配之间的矛盾推出 $s(N)t(N) = \Omega(N)$ ，因此上述稳定空间和临时空间的指数折中已经达到渐近最优。对标准实现，作者进一步把真实的块合并映射为 growth game，并利用二项式阈值证明：当稳定空间限制为 $N + O(rN^{1/r})$ 且参数满足相应条件时，Grow 的均摊代价至少为 $\Omega(r)$ 。这一结论与构造的 $O(r)$ 上界匹配，但其适用范围不能直接扩展到所有可能的非标准算法。

总体而言，该论文最重要的贡献不仅是给出一个更省稳定空间的数组实现，还在

于建立一套较完整的时间–空间边界：乘积下界刻画空间折中，分层构造给出可达上界，growth game 则解释标准块合并策略为何必须付出相应的更新时间。与此同时，word-RAM、常数时间位操作、均摊更新以及标准实现等假设也限制了结论的直接工程意义。如何把时间下界推广到一般算法、简化 growth game 的下界证明，并评估固定小参数结构的实际表现，仍是值得继续研究的问题。

参考文献

- [1] Robert E. Tarjan and Uri Zwick. Optimal resizable arrays, 2023.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.
- [3] Phil Bagwell. Fast functional lists. In *Implementation of Functional Languages*, volume 2670 of *Lecture Notes in Computer Science*, pages 34–50. Springer, 2003.
- [4] Michael T. Goodrich and John G. Kloss II. Tiered vectors: Efficient dynamic arrays for rank-based sequences. In *Algorithms and Data Structures*, volume 1663 of *Lecture Notes in Computer Science*, pages 205–216. Springer, 1999.
- [5] Philip Bille, Anders Roy Christiansen, Mikko Berggren Ettienne, and Inge Li Gørtz. Fast dynamic arrays. In *25th Annual European Symposium on Algorithms (ESA 2017)*, volume 87 of *Leibniz International Proceedings in Informatics*, pages 16:1–16:13. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2017.
- [6] Stelios Joannou and Rajeev Raman. An empirical evaluation of extendible arrays. In *Experimental Algorithms*, volume 6630 of *Lecture Notes in Computer Science*, pages 447–458. Springer, 2011.
- [7] Edward Sitarski. Algorithm alley: Hats: Hashed array trees. *Dr. Dobb’s Journal of Software Tools*, 21(9):107, 1996.
- [8] Andrej Brodnik, Svante Carlsson, Erik D. Demaine, J. Ian Munro, and Robert Sedgewick. Resizable arrays in optimal time and space. In *Proceedings of the 6th International Workshop on Algorithms and Data Structures (WADS)*, pages 37–48, 1999.